UNIVERSIDAD
NACIONAL
DE COLOMBIA

INGENIERÍA DE SOFTWARE II

Proyecto de Aula

Sistema de Gestión de Inventarios basado en Arquitectura de Microservicios

Entrega 1: Investigación y fundamentación

Caso de estudio: Distribuidora El Montañero S.A.S.

Presentado por

Luis Alejandro Saraza Marín
Brayan David Collazos Escobedo
Mateo Falla Muñoz

Presentado a

José Albeiro Montes Gil

Universidad Nacional de Colombia, Sede Manizales
Facultad de Administración
Departamento de Informática y Computación

Manizales, octubre de 2026

Índice

1. Introducción **1**

2. Investigación de sistemas similares **1**
2.1. InvenTree . . . 1
2.2. Part-DB . . . 2
2.3. Zoho Inventory . . . 3
2.4. Cuadro comparativo . . . 4
2.5. Conclusiones de la investigación . . . 5

3. Definición del problema y alcance **6**
3.1. Empresa caso de estudio . . . 6
3.2. Situación actual . . . 6
3.3. Planteamiento del problema . . . 7
3.4. Objetivo del sistema . . . 7
3.5. Actores y roles . . . 7
3.6. Alcance del sistema . . . 7
3.7. Fuera del alcance . . . 8

4. Justificación técnica de la arquitectura **8**
4.1. Microservicios frente a monolito . . . 8
4.2. Decisión . . . 9

5. Identificación preliminar de microservicios **9**
5.1. Contextos delimitados . . . 9
5.2. Vista preliminar de la arquitectura . . . 10
5.3. Mecanismo de comunicación . . . 10

6. Pila tecnológica **11**
6.1. Criterio de asignación . . . 11
6.2. Asignación por componente . . . 12
6.3. Herramientas transversales . . . 13
6.4. Flujo de autenticación . . . 13
6.5. Estrategia preliminar de despliegue . . . 13

7. Requerimientos **13**
7.1. Requerimientos funcionales . . . 13
7.2. Requerimientos no funcionales . . . 15

8. Repositorio y organización del equipo **16**
8.1. Repositorio en GitHub . . . 16

9. Bitácora de uso de IA **17**

Referencias **19**

1. Introducción

Este documento corresponde a la primera entrega del proyecto de aula del curso Ingeniería de Software II, desarrollado bajo la metodología de Aprendizaje Basado en Proyectos (ABP). En la fase de investigación y fundamentación, el equipo analiza tres sistemas de gestión de inventarios existentes, define el problema y el alcance del sistema para la empresa caso de estudio, justifica técnicamente la elección de una arquitectura de microservicios frente a una monolítica, identifica de forma preliminar los microservicios y su pila tecnológica, y especifica los requerimientos funcionales y no funcionales. El documento cierra con la bitácora de uso de inteligencia artificial de la fase.

El código fuente, la documentación y las bitácoras del proyecto se alojan en el repositorio del equipo:

`https://github.com/Mat-FM/Sistemas_Gestor_Inventario`

La Tabla 1 relaciona cada entregable exigido por el enunciado con la sección del documento que lo desarrolla.

Tabla 1. Entregables de la Entrega 1 y su ubicación en el documento

Entregable exigido por el enunciado	**Sección del documento**
Investigación de al menos tres sistemas similares, con comparativo de funcionalidades, fortalezas y debilidades	Sección 2
Definición del problema y alcance del sistema	Sección 3
Justificación técnica de microservicios frente a monolito	Sección 4
Identificación preliminar de los microservicios	Secciones 5 y 6
Documento de requerimientos funcionales y no funcionales	Sección 7
Repositorio de GitHub con actividad de todos los integrantes	Sección 8
Bitácora de uso de IA de la fase	Sección 9

2. Investigación de sistemas similares

Se analizaron tres sistemas representativos de distintos modelos de distribución: dos de código abierto y autoalojados (InvenTree y Part-DB) y uno comercial ofrecido como servicio en la nube (Zoho Inventory). Los datos de versiones, precios y funcionalidades corresponden a la documentación oficial consultada en septiembre de 2026.

2.1 InvenTree

InvenTree es un sistema de gestión de inventarios de código abierto que ofrece control de existencias de bajo nivel y seguimiento de partes. Su núcleo es un *backend* en Python con Django que expone una interfaz web y una API REST, y puede ampliarse mediante *plugins*. Sus desarrolladores lo describen como una herramienta liviana y fácil de usar para pymes y aficionados (InvenTree, s. f.-b).

Tabla 2. Ficha técnica de InvenTree

Aspecto	**Detalle**
Tipo y licencia	Código abierto (MIT), autoalojado
Versión estable	1.5.6, publicada el 26 de septiembre de 2026 (InvenTree, s. f.-i)
Tecnología	Python, Django y Django REST Framework en el *backend*; React en el *frontend* (InvenTree, s. f.-b)
Base de datos	PostgreSQL, MySQL o SQLite (InvenTree, s. f.-b)
API	REST con autenticación por *token* (InvenTree, s. f.-c)
Despliegue	Docker Compose o instalador para Linux (InvenTree, s. f.-a, s. f.-b)
Costo	Sin costo de licencia

Funcionalidades principales. Organiza los productos, a los que llama «partes», en categorías, con atributos, precios y unidades de medida. Controla las existencias por ubicación con ajustes, órdenes de transferencia, trazabilidad, historial de movimientos y control de vencimiento. En compras gestiona proveedores y órdenes de compra con recepción de mercancía (InvenTree, s. f.-h); en ventas, clientes, órdenes de venta y órdenes de devolución (InvenTree, s. f.-j). Incluye además manufactura con listas de materiales y órdenes de construcción, reportes y etiquetas basados en plantillas, y lectura de códigos de barras (InvenTree, s. f.-g).

Cada parte puede tener un stock mínimo: cuando la existencia total cae por debajo de ese valor, la parte se marca como de stock bajo en la interfaz (InvenTree, s. f.-g), y los usuarios suscritos a la parte o a su categoría reciben una notificación por correo electrónico (InvenTree, s. f.-f). Una tarea diaria en segundo plano avisa también sobre existencias próximas a vencer (InvenTree, s. f.-k).

Fortalezas.

- Licencia MIT y API REST documentada como base para integraciones (InvenTree, s. f.-b, s. f.-c).
- Procesamiento asíncrono: un *worker* dedicado ejecuta las tareas que no conviene atender en el servidor web (InvenTree, s. f.-d).
- Alertas por stock mínimo y por vencimiento (InvenTree, s. f.-f, s. f.-k).
- Desarrollo activo: en 2026 publicó las versiones 1.2, 1.3, 1.4 y 1.5 (InvenTree, s. f.-e).

Debilidades.

- Operación compleja: la instalación con Docker levanta cinco contenedores (base de datos, servidor web, *worker*, *proxy* y caché) (InvenTree, s. f.-a).
- Orientado a manufactura: buena parte de sus módulos (listas de materiales, órdenes de construcción) no aplica a una distribuidora.

2.2 Part-DB

Part-DB es un sistema de inventarios de código abierto orientado a componentes electrónicos, que se instala en un servidor web y se usa desde el navegador. Lo emplean aficionados y *makerspaces*, pero también pequeñas empresas y universidades (Part-DB, s. f.-f). Su concepto central es la «parte», un tipo de artículo cuyas existencias se distribuyen en lotes, cada uno con una ubicación y una cantidad (Part-DB, s. f.-b).

Tabla 3. Ficha técnica de Part-DB

Aspecto	**Detalle**
Tipo y licencia	Código abierto (AGPL-3.0), autoalojado (Part-DB, s. f.-f)
Versión estable	2.19.0, publicada el 27 de septiembre de 2026 (Part-DB, s. f.-e)
Tecnología	PHP 8.2 o superior con Symfony; Bootstrap 5 y Hotwire en el *frontend* (Part-DB, s. f.-f)
Base de datos	MySQL, MariaDB, PostgreSQL o SQLite (Part-DB, s. f.-f)
API	REST con *tokens* de alcance limitado y servidor OAuth 2.0 (Part-DB, s. f.-a, s. f.-d)
Despliegue	Servidor web, imagen Docker o Kubernetes mediante Helm (Part-DB, s. f.-c, s. f.-f)
Costo	Sin costo de licencia

Funcionalidades principales. Cada parte se asocia a una categoría, un *footprint*, un fabricante, varias ubicaciones e información de precios, y admite adjuntos como hojas de datos o imágenes. El sistema maneja usuarios y grupos con permisos de granularidad fina, autenticación de dos factores e inicio de sesión único mediante SAML. Su registro de eventos guarda qué cambios hace cada usuario y permite revertir partes a versiones anteriores. Ofrece búsqueda paramétrica, generación de etiquetas con códigos de barras, escaneo mediante cámara web, importación y exportación de datos, e integración con KiCad y con proveedores en línea como Digikey o LCSC (Part-DB, s. f.-f). Sin embargo, el campo de stock mínimo figura en la documentación como no implementado por completo (Part-DB, s. f.-b).

Fortalezas.

- Seguridad: permisos granulares por grupo y doble factor de autenticación (Part-DB, s. f.-f).
- Auditoría: registro de eventos con reversión de cambios (Part-DB, s. f.-f).
- Despliegue sencillo, con soporte oficial para Kubernetes (Part-DB, s. f.-c).
- Interfaz *responsive*, utilizable desde computador, tableta o celular (Part-DB, s. f.-f).

Debilidades.

- Dominio especializado en componentes electrónicos (Part-DB, s. f.-f).
- Su documentación no describe órdenes de compra, devoluciones ni reportes de inventario, y el stock mínimo está incompleto (Part-DB, s. f.-b, s. f.-f).
- El español no figura entre los idiomas soportados (Part-DB, s. f.-f).

2.3 Zoho Inventory

Zoho Inventory es un software comercial de gestión de inventarios que Zoho Corporation ofrece como servicio en la nube y que hace parte de su familia de aplicaciones Zoho Finance, junto con Zoho Books, Zoho Billing y Zoho Commerce, entre otras (Zoho Corporation, s. f.-g).

Tabla 4. Ficha técnica de Zoho Inventory

Aspecto	Detalle
Tipo	SaaS comercial propietario
Costo (edición EE. UU.)	Plan gratuito (50 órdenes al mes, 1 usuario, 1 bodega) y planes pagos entre USD 29 y USD 249 mensuales por organización, con facturación anual (Zoho Corporation, s. f.-c)
Usuarios por plan	1 (gratuito), 3 (Standard), 5 (Premium) y 10 (Plus y Enterprise) (Zoho Corporation, s. f.-c)
Plataformas	Web y aplicaciones para iOS y Android (Zoho Corporation, s. f.-h, s. f.-i)
API	REST con OAuth 2.0; límite de 100 peticiones por minuto por organización (Zoho Corporation, s. f.-b)
Infraestructura	Gestionada por Zoho en ocho dominios regionales (Zoho Corporation, s. f.-b)

Funcionalidades principales. Gestiona artículos, grupos y artículos compuestos, niveles de reorden, seguimiento por lote y por número de serie, ajustes, conteos de stock y códigos de barras, con varias bodegas y órdenes de transferencia (Zoho Corporation, s. f.-c). Al activar la notificación de reorden, el sistema envía un correo cuando un artículo cae por debajo de su nivel de reorden (Zoho Corporation, s. f.-a); además, el módulo de reabastecimiento lista los artículos con stock bajo y facilita crear la orden de compra o de transferencia correspondiente (Zoho Corporation, s. f.-e). Sus reportes incluyen resumen y antigüedad del inventario, valorización por FIFO o costo promedio ponderado y clasificación ABC (Zoho Corporation, s. f.-d, s. f.-f). El acceso se controla con roles predefinidos y personalizados (Zoho Corporation, s. f.-g).

Fortalezas.

- Es el más completo de los tres sistemas en compras, reabastecimiento y reportes (Zoho Corporation, s. f.-d, s. f.-e).
- No exige infraestructura propia y cuenta con aplicaciones móviles nativas (Zoho Corporation, s. f.-h, s. f.-i).
- API documentada, con OAuth 2.0 y límites de uso explícitos (Zoho Corporation, s. f.-b).
- Documentación oficial disponible en español (Zoho Corporation, s. f.-d).

Debilidades.

- Pago recurrente y límites de órdenes, usuarios y bodegas según el plan (Zoho Corporation, s. f.-c).
- Código cerrado y dependencia directa del proveedor.
- Ninguno de sus dominios regionales está en Latinoamérica: Estados Unidos, Europa, India, Australia, Japón, Canadá, Arabia Saudita y Reino Unido (Apideck, s. f.).

2.4 Cuadro comparativo

La Tabla 5 consolida las tres dimensiones exigidas por el enunciado: funcionalidades, arquitectura y experiencia de usuario.

Tabla 5. Comparativo de los sistemas analizados

Criterio	InvenTree	Part-DB	Zoho Inventory
Funcionalidades			
Catálogo y stock	Partes, categorías, existencias por ubicación	Partes y lotes por ubicación	Artículos, lotes, series y varias bodegas
Órdenes de compra	Sí, con recepción	No documentadas	Sí, con reabastecimiento
Devoluciones	Órdenes de devolución	No documentadas	Sí
Reportes	Plantillas configurables	No documentados	Valorización FIFO y promedio, ABC, antigüedad
Alertas de stock	Correo a usuarios suscritos	Stock mínimo incompleto	Correo al llegar al nivel de reorden
Vencimientos	Aviso diario de stock próximo a vencer	No documentado	Seguimiento por lotes
Usuarios y permisos	Permisos por usuario	Grupos con permisos granulares y 2FA	Roles predefinidos y personalizados
Arquitectura			
Licencia	MIT	AGPL-3.0	Propietaria, por suscripción
Infraestructura	Autoalojada	Autoalojada	Nube de Zoho
Estilo	Monolito con *worker* en segundo plano	Monolito (Symfony)	No publicado
Despliegue	Docker Compose con cinco contenedores	Imagen Docker o Helm en Kubernetes	Gestionado por el proveedor
API	REST con *tokens*	REST con *tokens* y OAuth 2.0	REST con OAuth 2.0 y límite de tasa
Persistencia	Una base de datos para toda la aplicación	Una base de datos para toda la aplicación	Gestionada por el proveedor
Experiencia de usuario			
Idioma español	Traducción comunitaria	No soportado	Documentación oficial en español
Plataformas	Web y app móvil complementaria	Web *responsive*	Web y apps iOS y Android
Códigos de barras	Sí	Sí, con cámara web	Sí
Puesta en marcha	Compleja (varios servicios)	Media (un contenedor)	Inmediata (registro en línea)

2.5 Conclusiones de la investigación

1. **Existe un núcleo funcional común**: productos, existencias por ubicación, proveedores, usuarios con permisos, búsqueda y una API. Es la línea base del alcance del sistema propio.
2. **Compras, devoluciones y reportes distinguen una solución completa de una básica.** Zoho Inventory es el referente en estas áreas; de él se toman la recepción contra orden de compra, el reabastecimiento a partir del nivel de reorden y la valorización del inventario.
3. **Hay un espacio poco cubierto en la distribución de productos de consumo.** Las opciones de código

abierto están pensadas para manufactura y electrónica (InvenTree, s. f.-b; Part-DB, s. f.-f), y ninguna de las tres propone la salida por lote más próximo a vencer (FEFO), que es esencial para una distribuidora de productos empacados.

4. **Contenerizar no equivale a tener microservicios.** Part-DB es un monolito y aun así se despliega en contenedores y en Kubernetes (Part-DB, s. f.-c). Por eso la justificación de la arquitectura propia se apoya en el escalado y el despliegue independientes y en el aislamiento de fallos. La separación del *worker* asíncrono de InvenTree (InvenTree, s. f.-d) se adopta como patrón para notificaciones y reportes.
5. **La seguridad se concentra en el punto de entrada de la API.** Los tres sistemas protegen su API con *tokens* u OAuth 2.0, y Zoho limita además las peticiones por organización y responde con HTTP 429 al superar el umbral (Zoho Corporation, s. f.-b). Ese esquema orienta el diseño del API Gateway propio (RNF09 y RNF19).

3. Definición del problema y alcance

3.1 Empresa caso de estudio

Distribuidora El Montañero S.A.S. es una empresa ficticia ubicada en Manizales, Caldas, que distribuye insumos y productos empacados a tiendas de barrio y cafeterías del Eje Cafetero. Opera con dos bodegas, la principal en Manizales y una satélite en Chinchiná; maneja alrededor de 800 referencias de unas 40 marcas y atiende cerca de 150 clientes. Su equipo está conformado por seis personas: un administrador, dos auxiliares de bodega, dos vendedores y un contador externo.

Se eligió esta empresa porque su tamaño hace visible el problema sin volverlo inabarcable: tiene más de una ubicación física, alta rotación de mercancía, productos con fecha de vencimiento y un equipo pequeño que no puede dedicar a nadie de tiempo completo a cuadrar el inventario.

3.2 Situación actual

El control de inventario se lleva en un archivo de Excel compartido por correo, acompañado de un cuaderno de entradas y salidas en cada bodega. De este esquema se derivan seis problemas:

1. **No hay una única versión de la información.** Cada bodega actualiza su propia copia del archivo y la consolidación se hace una vez por semana; durante seis días nadie conoce el saldo real.
2. **Los vendedores comprometen mercancía inexistente.** Consultan una hoja desactualizada y prometen productos ya vendidos, lo que genera pedidos incompletos y devoluciones.
3. **Las compras se deciden por intuición.** Sin un histórico confiable de consumo, se sobreabastecen referencias de baja rotación mientras se agotan las de alta.
4. **No hay trazabilidad.** Cuando falta mercancía no se sabe quién la movió ni cuándo; los faltantes aparecen en el conteo anual y se asumen como pérdida.
5. **Los vencimientos se detectan tarde.** Los productos próximos a vencer se identifican al revisar físicamente el estante.
6. **Los traslados entre bodegas quedan sin registro.** La mercancía que sale de Manizales no figura en ningún saldo hasta que alguien la registra en Chinchiná, a veces días después.

3.3 Planteamiento del problema

El Montañero no dispone de información de inventario confiable, oportuna ni trazable. Esto produce ventas comprometidas sin respaldo real, compras mal dimensionadas, pérdidas por vencimiento y faltantes sin responsable identificable. El control manual actual no escala con el crecimiento en número de referencias, bodegas y usuarios que trabajan de forma simultánea.

3.4 Objetivo del sistema

Construir un sistema web de gestión de inventarios, basado en una arquitectura de microservicios, que centralice el saldo de existencias por producto y bodega en tiempo real, registre cada movimiento con su responsable, anticipe los quiebres de stock y los vencimientos, y entregue indicadores que sustenten las decisiones de compra.

3.5 Actores y roles

Tabla 6. Actores del sistema y permisos generales

Rol	Quién lo ocupa	Permisos principales
Administrador	Administrador de la empresa	Acceso total: usuarios, catálogo, proveedores, órdenes de compra, umbrales, reportes y configuración.
Operador de bodega	Auxiliares de bodega	Registrar entradas, salidas, ajustes, traslados, devoluciones, recepciones y conteos físicos en su bodega.
Vendedor	Vendedores	Consulta del catálogo y de las existencias por bodega en tiempo real.
Contador	Contador externo	Lectura del kardex, del panel de auditoría y de la valorización; exportación de reportes.

3.6 Alcance del sistema

El sistema incluye:

- Catálogo de productos, categorías y proveedores.
- Control de existencias por producto y bodega: entradas, salidas, ajustes y traslados con estado *en tránsito*.
- Gestión de varias bodegas y consulta de existencias por ubicación.
- Órdenes de compra a proveedores con recepción de mercancía total o parcial.
- Usuarios con los roles de la Tabla 6 y autenticación.
- Alertas por stock bajo el umbral mínimo y por productos próximos a vencer.
- Kardex completo: quién, cuándo, qué producto y qué tipo de movimiento.
- Reportes de existencias, valorización y rotación, con panel de indicadores.
- Conteo físico programado con generación automática de los ajustes.
- Carga inicial de productos y saldos desde archivos Excel o CSV.

- Devoluciones de clientes y hacia proveedores, reflejadas en las existencias.
- Interfaz *responsive*, utilizable desde tableta o celular dentro de la bodega.

3.7 Fuera del alcance

Para que el proyecto sea realizable en un semestre, quedan excluidos:

- Facturación electrónica, integración con la DIAN, cartera de clientes y contabilidad general.
- Gestión de pedidos de venta y despachos; las salidas se registran como movimientos de inventario con un documento de referencia.
- Aplicaciones móviles nativas; el acceso móvil se resuelve con la interfaz web *responsive*.
- Integración con tiendas en línea, *marketplaces* o pasarelas de pago.
- Lectores de código de barras dedicados y predicción de demanda con modelos estadísticos.

4. Justificación técnica de la arquitectura

4.1 Microservicios frente a monolito

El sistema comprende dominios de negocio diferenciados (catálogo, inventario, proveedores, compras, usuarios, notificaciones y reportes) con perfiles de uso distintos: la consulta de existencias es frecuente y concurrente, mientras que los reportes son esporádicos pero costosos. La Tabla 7 compara ambas arquitecturas con los criterios técnicos del enunciado, aplicados al caso de El Montañero, e incluye los criterios en los que el monolito resulta superior.

Tabla 7. Comparación de arquitecturas para el caso de estudio

Criterio	**Arquitectura monolítica**	**Arquitectura de microservicios**
Escalabilidad	Se escala la aplicación completa aunque solo la consulta de existencias tenga carga alta.	Se escala solo el servicio con más demanda, por ejemplo `ms-inventario` durante la jornada de ventas.
Mantenibilidad	Un cambio en un módulo puede afectar a otros por dependencias internas poco visibles.	Cada servicio tiene un dominio acotado y un contrato de API; modificar el catálogo no altera el kardex.
Despliegue independiente	Actualizar una función exige redesplegar todo el sistema y detener la operación en ambas bodegas.	Se actualiza un servicio sin interrumpir los demás, por ejemplo cambiar reportes durante un conteo físico.
Tolerancia a fallos	Un error grave, como un reporte que agota la memoria, puede detener el proceso completo y bloquear el registro de mercancía.	Los fallos quedan aislados: si fallan notificaciones o reportes, se siguen registrando movimientos (RNF18).
Trabajo en equipo	Los tres integrantes trabajan sobre la misma base de código, con más conflictos de integración.	Cada integrante es responsable de servicios concretos y trabaja en paralelo sobre contratos acordados.
Persistencia	Base única y compartida; las consultas de reportes compiten con las transacciones de stock.	Base de datos por servicio (RNF02); reportes usa su propio modelo de lectura sin cargar la base de inventario.

Criterio	Arquitectura monolítica	Arquitectura de microservicios
Complejidad operativa	Ventaja del monolito: un único artefacto que desplegar y monitorear.	Más componentes: red, imágenes, manifiestos y registros distribuidos. Se mitiga con Kubernetes y Docker Compose en desarrollo.
Consistencia de datos	Ventaja del monolito: transacciones ACID entre módulos.	Consistencia eventual entre servicios. Se mitiga con idempotencia (RNF17) y eventos con reintento.
Latencia	Ventaja del monolito: llamadas en memoria.	Saltos de red entre servicios. Se mitiga con tiempos de espera, pocas llamadas por operación y el criterio de 2 s (RNF06).

4.2 Decisión

Con seis usuarios actuales, un monolito modular sería técnicamente viable. Aun así, se elige la arquitectura de microservicios por cuatro razones concretas. Primera, los dominios tienen perfiles de carga muy distintos, lo que hace valioso el escalado independiente de la consulta de existencias. Segunda, el aislamiento de fallos protege la operación crítica, el registro de movimientos, frente a componentes no críticos como notificaciones y reportes. Tercera, la empresa proyecta crecer en bodegas y usuarios, y cada servicio puede evolucionar sin redesplegar el sistema completo. Cuarta, permite que los tres integrantes desarrollen en paralelo. Los costos reconocidos en la Tabla 7 se asumen de forma consciente y se mitigan con los requerimientos RNF10, RNF17 y RNF18.

5. Identificación preliminar de microservicios

5.1 Contextos delimitados

Cada microservicio es dueño exclusivo de sus datos y ningún otro servicio accede a su base de datos (RNF02). La Tabla 8 asigna cada requerimiento funcional a un servicio responsable.

Tabla 8. Microservicios, responsabilidades y requerimientos asignados

Servicio	Responsabilidad	Datos que posee	Requerimientos
`api-gateway`	Único punto de entrada: enrutamiento, validación de JWT, control de roles, límite de tasa e identificador de correlación.	Ninguno	RNF09, RNF19
`ms-usuarios`	Usuarios, roles, inicio y cierre de sesión, emisión de JWT, recuperación y cambio de contraseña.	Usuario, Rol, TokenRecuperación	RF07, RF08
`ms-catalogo`	Productos, categorías y subcategorías; búsqueda y filtrado.	Producto, Categoría	RF01, RF02, RF12
`ms-proveedores`	Proveedores y su asociación con productos.	Proveedor, ProveedorProducto	RF03

Servicio	Responsabilidad	Datos que posee	Requerimientos
`ms-inventario`	Bodegas, saldos, movimientos, traslados, lotes con salida FEFO, kardex, devoluciones de clientes y conteo físico. Detecta el quiebre de umbral que origina las alertas.	Bodega, Saldo, Movimiento, Lote, Traslado, Conteo	RF04, RF05, RF09, RF10, RF13, RF15, RF17, RF18
`ms-compras`	Órdenes de compra, cambios de estado, recepción total o parcial y devoluciones a proveedor.	OrdenCompra, DetalleOrden, Recepción	RF06, RF13
`ms-notificaciones`	Entrega de alertas en la interfaz y por correo; revisión diaria de vencimientos.	Notificación, Preferencia	RF09, RF17
`ms-reportes`	Reportes, panel de indicadores, exportación a CSV y PDF, e importación masiva validada.	Modelo de lectura de movimientos y saldos, Importación	RF11, RF14, RF15, RF16

5.2 Vista preliminar de la arquitectura

La Figura 1 presenta los componentes y sus relaciones. El diagrama detallado, con contratos de API y modelos de datos, corresponde a la Entrega 2.

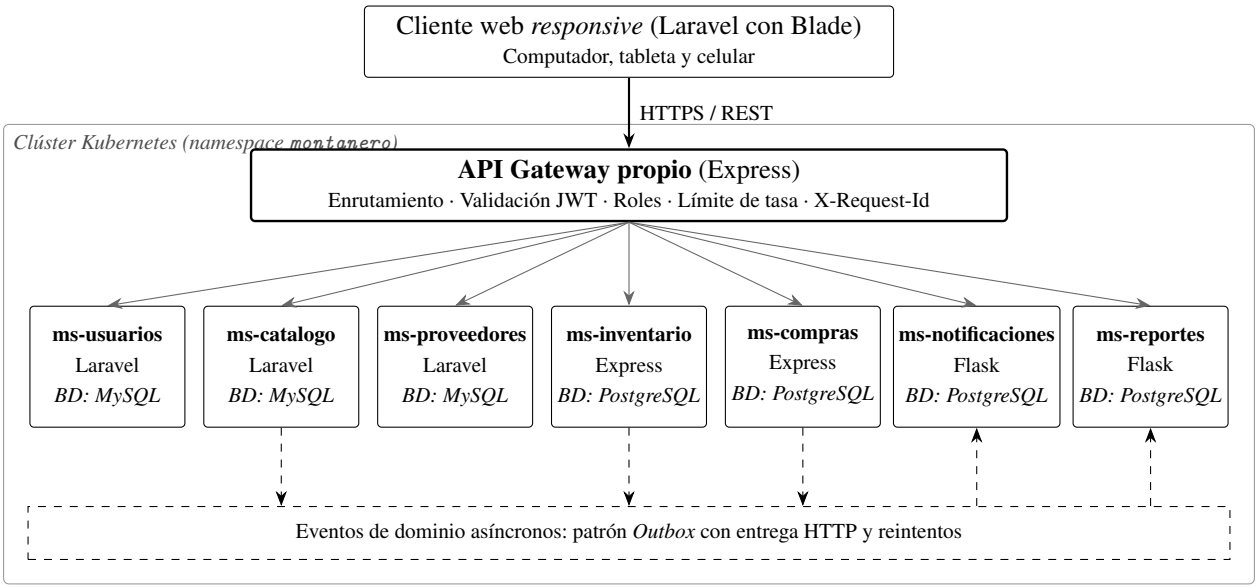


Figura 1. Vista preliminar de componentes. Las líneas continuas representan peticiones REST síncronas; las discontinuas, la publicación y el consumo de eventos.

5.3 Mecanismo de comunicación

Se adopta un esquema híbrido:

- **Comunicación síncrona (REST con JSON)** para operaciones que requieren respuesta inmediata. Todas las peticiones externas pasan por el gateway, y entre servicios solo se hacen llamadas puntuales: `ms-inventario` valida que el producto exista en `ms-catalogo`, y `ms-compras` registra la entrada en `ms-inventario` al recibir mercancía, con clave de idempotencia (RNF17). El descubrimiento de servicios lo resuelve el DNS interno de Kubernetes, por ejemplo `http://ms-inventario:3000`.

- **Comunicación asíncrona (eventos)** para efectos secundarios que no deben bloquear la operación principal. Como en el curso no se trabajó un *broker* de mensajería, se aplica el patrón *Outbox* transaccional: el servicio productor guarda el evento en una tabla `outbox` dentro de la misma transacción del movimiento, y un proceso interno lo entrega por HTTP a los consumidores con reintentos. Los consumidores descartan eventos repetidos por su identificador. Así, si un consumidor está caído, el movimiento se registra igualmente y el evento se entrega cuando el consumidor se recupera.

Tabla 9. Eventos de dominio preliminares

Evento	Productor	Consumidores	Propósito
`MovimientoRegistrado`	`ms-inventario`	`ms-reportes`	Actualizar el modelo de lectura de rotación y valorización.
`StockBajoUmbral`	`ms-inventario`	`ms-notificaciones`	Disparar la alerta de RF09.
`OrdenCompraRecibida`	`ms-compras`	`ms-reportes`, `ms-notificaciones`	Indicadores de compras y aviso al administrador.
`ProductoActualizado`	`ms-catalogo`	`ms-reportes`	Mantener nombre y precio para la valorización.

6. Pila tecnológica

6.1 Criterio de asignación

De acuerdo con RNF15, los servicios se construyen exclusivamente con los tres *frameworks* trabajados en clase, y la persistencia usa los dos motores de base de datos vistos en el curso, MySQL y PostgreSQL. La asignación aprovecha la fortaleza de cada *framework*:

- **Laravel (PHP)** para dominios CRUD con reglas de negocio y seguridad. Su ORM Eloquent, la validación de solicitudes, las migraciones, el cifrado de contraseñas con bcrypt y el flujo nativo de recuperación de contraseña por correo reducen el código repetitivo.
- **Express (Node.js)** para los componentes de alta concurrencia y entrada/salida intensiva. Su modelo no bloqueante es adecuado para el gateway, que solo reenvía peticiones, y para la consulta de existencias, que es el servicio a escalar (RNF05).
- **Flask (Python)** para procesamiento de datos y tareas programadas. Bibliotecas como pandas, openpyxl y reportlab facilitan las agregaciones, la lectura de Excel y CSV, y la generación de PDF.

6.2 Asignación por componente

Tabla 10. Pila tecnológica por componente

Componente	Framework	Justificación	Base de datos	Bibliotecas principales
`api-gateway`	Express	Proxy liviano y no bloqueante; *middlewares* encadenables para JWT, roles, límite de tasa y registro.	Ninguna	jsonwebtoken, express-rate-limit, axios, helmet, cors
`ms-usuarios`	Laravel	bcrypt nativo, recuperación de contraseña con *tokens* y correo, políticas de autorización.	MySQL	firebase/php-jwt, L5-Swagger
`ms-catalogo`	Laravel	CRUD con validaciones, paginación y filtros combinables (RF12).	MySQL	Eloquent, L5-Swagger
`ms-proveedores`	Laravel	CRUD con relación muchos a muchos entre proveedor y producto.	MySQL	Eloquent, L5-Swagger
`ms-inventario`	Express	Servicio más consultado; transacciones con bloqueo de fila para evitar saldos negativos en movimientos concurrentes.	PostgreSQL	pg, Knex, swagger-jsdoc, swagger-ui-express
`ms-compras`	Express	Flujo de estados de la orden y coordinación de la recepción con inventario.	PostgreSQL	pg, Knex, swagger-ui-express
`ms-notificaciones`	Flask	Envío de correo y tarea diaria de revisión de vencimientos.	PostgreSQL	Flask-Mail, APScheduler, flask-smorest
`ms-reportes`	Flask	Agregaciones de rotación y valorización; lectura y validación de Excel y CSV; exportación a PDF.	PostgreSQL	pandas, openpyxl, reportlab, flask-smorest
Cliente web	Laravel	Interfaz *responsive* con Blade que consume exclusivamente el gateway.	Ninguna	Blade, Bootstrap

El enrutamiento, la validación de *tokens* y el límite de tasa del gateway se programan por el equipo como *middlewares* de Express. No se usan soluciones de gateway preconstruidas como Kong ni el paquete Express Gateway, que también es un producto de terceros (RNF09).

6.3 Herramientas transversales

Tabla 11. Herramientas de infraestructura, calidad y pruebas

Necesidad	Herramienta
Contenedores	Docker, con un Dockerfile por servicio e imágenes construidas por etapas; Docker Compose solo en desarrollo local.
Orquestación	Kubernetes (Minikube en desarrollo): Deployments, Services, StatefulSets con volúmenes persistentes, ConfigMaps, Secrets, HorizontalPodAutoscaler y NetworkPolicies.
Documentación de API	Especificación OpenAPI 3 por servicio, publicada en la ruta `/docs`.
Pruebas de rendimiento	k6, porque sus escenarios se escriben en JavaScript, lenguaje que el equipo ya usa con Express, y reporta directamente percentiles de tiempo de respuesta, tasa de error y *throughput*.
Pruebas unitarias	PHPUnit (Laravel), Jest con Supertest (Express) y pytest (Flask).
Integración continua	GitHub Actions, con pruebas y análisis estático en cada *pull request* (RNF20).
Control de versiones	Git y GitHub, con ramas por funcionalidad y *pull requests* revisados.

6.4 Flujo de autenticación

1. El cliente envía sus credenciales a `POST /api/v1/auth/login`, y el gateway enruta la petición a `ms-usuarios`.
2. `ms-usuarios` verifica la contraseña cifrada con bcrypt y emite un JWT firmado que contiene el identificador, los roles y una expiración de 60 minutos.
3. En cada petición posterior, el gateway valida la firma y la expiración, comprueba el rol que exige la ruta y reenvía la solicitud con las cabeceras `X-User-Id`, `X-User-Roles` y `X-Request-Id`.
4. Los microservicios no se exponen fuera del clúster (Services de tipo ClusterIP y NetworkPolicies), de modo que solo reciben tráfico que ya pasó por el gateway.

6.5 Estrategia preliminar de despliegue

Cada microservicio tendrá un Deployment con sondas de disponibilidad y de vida sobre la ruta `/health`, y un Service de tipo ClusterIP. Cada base de datos se desplegará como StatefulSet con su propio PersistentVolumeClaim y credenciales en un Secret independiente. El gateway será el único Service expuesto al exterior. Se configurará un HorizontalPodAutoscaler para `ms-inventario` y `api-gateway`, de una a cuatro réplicas con un objetivo de 70 % de uso de CPU. Los manifiestos detallados se presentan en la Entrega 2.

7. Requerimientos

7.1 Requerimientos funcionales

Los requerimientos RF01 a RF14 corresponden al alcance mínimo del enunciado; RF15 a RF18 son ampliaciones del equipo derivadas del caso de estudio. La prioridad se clasifica como alta (A), media (M) o baja (B).

Tabla 12. Requerimientos funcionales

Código	Requerimiento	Descripción	Servicio	Prior.
RF01	Gestión de productos	Crear, consultar, actualizar y dar de baja productos con código, nombre, descripción, categoría, unidad de medida, precio y proveedor asociado.	`ms-catalogo`	A
RF02	Gestión de categorías	Crear, consultar, actualizar y eliminar categorías y subcategorías.	`ms-catalogo`	A
RF03	Gestión de proveedores	Registrar, consultar, actualizar y eliminar proveedores, y asociarlos a uno o varios productos.	`ms-proveedores`	A
RF04	Control de stock	Registrar entradas, salidas, ajustes y traslados con estado en tránsito, y mantener el saldo por producto y bodega.	`ms-inventario`	A
RF05	Bodegas	Administrar varias bodegas y consultar el stock de un producto por ubicación.	`ms-inventario`	A
RF06	Órdenes de compra	Generar, consultar y cambiar el estado de órdenes de compra, y recibir mercancía total o parcial contra ellas.	`ms-compras`	A
RF07	Usuarios y roles	Registrar, consultar, actualizar y asignar usuarios con los roles de la Tabla 6.	`ms-usuarios`	A
RF08	Autenticación	Inicio y cierre de sesión, recuperación y cambio de contraseña.	`ms-usuarios`	A
RF09	Alertas de stock	Notificar en la interfaz y por correo cuando el stock de un producto quede por debajo de un umbral configurable.	`ms-inventario`, `ms-notificaciones`	A
RF10	Trazabilidad (kardex)	Registrar quién, cuándo, qué producto, en qué bodega y qué tipo de movimiento se realizó.	`ms-inventario`	A
RF11	Reportes	Existencias actuales, valorización del inventario y productos de mayor y menor rotación.	`ms-reportes`	A
RF12	Búsqueda y filtrado	Buscar y filtrar productos por nombre, código, categoría o proveedor.	`ms-catalogo`	A
RF13	Devoluciones	Registrar devoluciones de clientes (entradas) y hacia proveedores (salidas), reflejadas en el stock.	`ms-inventario`, `ms-compras`	A
RF14	Panel de indicadores	Indicadores de rotación, quiebres de stock, valor total del inventario y productos próximos a vencer.	`ms-reportes`	A
RF15	Panel de auditoría	Consultar el kardex filtrando por usuario, fecha, bodega, producto y tipo de movimiento, con exportación del resultado.	`ms-inventario`, `ms-reportes`	M
RF16	Carga y descarga masiva	Importar productos, proveedores y stock inicial desde CSV o Excel con validación previa y reporte de errores por fila; exportar reportes a CSV y PDF.	`ms-reportes`	M
RF17	Lotes, vencimiento y FEFO	Registrar lote y fecha de vencimiento, sugerir la salida del lote más próximo a vencer y alertar con una anticipación configurable en días.	`ms-inventario`, `ms-notificaciones`	A
RF18	Conteo físico	Programar conteos por bodega, registrar las cantidades contadas y generar los ajustes por diferencia con aprobación del administrador.	`ms-inventario`	B

7.2 Requerimientos no funcionales

Cada requerimiento no funcional se acompaña de un criterio verificable, para que pueda comprobarse en las pruebas y en la sustentación.

Tabla 13. Requerimientos no funcionales y criterios de verificación

Código	Requerimiento	Criterio de verificación
RNF01	Arquitectura	Siete microservicios y un gateway independientes; comunicación REST síncrona y eventos asíncronos mediante el patrón *Outbox*.
RNF02	Persistencia independiente	Una base de datos y un Secret de credenciales por servicio; ningún servicio conoce las credenciales de otro.
RNF03	Contenerización	Un Dockerfile por servicio; las bases de datos también se ejecutan en contenedores.
RNF04	Orquestación	Todo el sistema se despliega en Kubernetes con `kubectl apply` sobre los manifiestos del repositorio.
RNF05	Escalabilidad	HorizontalPodAutoscaler en `ms-inventario` y `api-gateway`, de una a cuatro réplicas con objetivo de 70 % de CPU; servicios sin estado en memoria.
RNF06	Rendimiento	Consultas (GET) con percentil 95 inferior a 2 s y tasa de error inferior a 1 % con 50 usuarios virtuales concurrentes durante 5 minutos. La cifra supera ocho veces la planta actual de seis usuarios para cubrir crecimiento y picos.
RNF07	Seguridad	JWT firmado con expiración de 60 minutos; contraseñas cifradas con bcrypt; *tokens* de recuperación de un solo uso con vigencia de 30 minutos.
RNF08	Documentación de API	Especificación OpenAPI 3 de cada servicio disponible en `/docs`.
RNF09	API Gateway propio	El gateway es el único Service expuesto fuera del clúster; los demás son ClusterIP con NetworkPolicies. No se usan Kong ni gateways de terceros.
RNF10	Observabilidad	Registros en formato JSON por salida estándar, con servicio, nivel e identificador `X-Request-Id`, consultables con `kubectl logs`; ruta `/health` en cada servicio.
RNF11	Pruebas de rendimiento	Quince pruebas por integrante (45 en total) en escenarios de carga normal, pico y estrés, con métricas y resultados documentados.
RNF12	Control de versiones	Commits, ramas y *pull requests* de los tres integrantes visibles en el repositorio de GitHub.
RNF13	Mantenibilidad	Separación en capas (rutas, controladores, servicios y acceso a datos) y análisis estático por lenguaje en la integración continua.
RNF14	Portabilidad	Toda la configuración se inyecta por variables de entorno; ningún valor dependiente del entorno queda en el código fuente.
RNF15	Tecnologías permitidas	Laravel, Flask y Express como *frameworks*; MySQL y PostgreSQL como bases de datos.
RNF16	Uso documentado de IA	Bitácora completa en cada una de las cuatro entregas.
RNF17	Idempotencia	Los endpoints de movimientos aceptan la cabecera `Idempotency-Key`; un reintento con la misma clave devuelve la respuesta original sin duplicar el movimiento.

Código	Requerimiento	Criterio de verificación
RNF18	Resiliencia	Tiempo de espera de 3 s en llamadas entre servicios; hasta tres reintentos con retroceso exponencial (0,2, 0,4 y 0,8 s) solo en operaciones idempotentes; cortacircuitos que se abre tras cinco fallos consecutivos y vuelve a probar a los 30 s. El registro de movimientos sigue operando aunque notificaciones o reportes no estén disponibles.
RNF19	Límite de tasa	100 peticiones por minuto por usuario autenticado y 10 intentos por minuto por dirección IP en el inicio de sesión; al exceder el límite se responde HTTP 429 con la cabecera `Retry-After`.
RNF20	Pruebas en integración continua	Pruebas unitarias ejecutadas en cada *pull request*, con cobertura mínima del 60 % en `ms-inventario`, `ms-usuarios` y `ms-catalogo`.
RNF21	Usabilidad en dispositivos móviles	Interfaz operable en pantallas desde 360 píxeles de ancho, para su uso con celular o tableta en bodega.

8. Repositorio y organización del equipo

8.1 Repositorio en GitHub

El proyecto se aloja en el repositorio `https://github.com/Mat-FM/Sistemas_Gestor_Inventario`, en el cual se registra la actividad de cada integrante mediante commits, ramas y *pull requests*. El repositorio se organiza como un monorepositorio con la siguiente estructura:

```
Sistemas_Gestor_Inventario/
|-- api-gateway/            Express
|-- services/
|   |-- ms-usuarios/        Laravel
|   |-- ms-catalogo/        Laravel
|   |-- ms-proveedores/     Laravel
|   |-- ms-inventario/      Express
|   |-- ms-compras/         Express
|   |-- ms-notificaciones/  Flask
|   ‘-- ms-reportes/        Flask
|-- web-client/             Laravel
|-- k8s/                    Manifiestos de Kubernetes
|-- performance-tests/      Pruebas k6 por integrante
|-- docs/                   Documentos de cada entrega
‘-- bitacoras-ia/           Bitácoras de uso de IA
```

El trabajo en el repositorio sigue estas convenciones:

- Ramas `main` (versión estable), `develop` (integración) y `feature/<servicio>-<descripción>` para cada funcionalidad.
- Todo cambio se integra mediante *pull request* hacia `develop`, revisado por otro integrante.
- Los mensajes de commit siguen la convención *Conventional Commits* (`feat:`, `fix:`, `docs:`).
- Cada integrante mantiene sus pruebas de rendimiento en `performance-tests/<nombre>/`.

9. Bitácora de uso de IA

Proyecto	Sistema de Gestión de Inventarios – Distribuidora El Montañero S.A.S.
Equipo / Integrantes	Luis Alejandro Saraza Marín, Brayan David Collazos Escobedo, Mateo Falla Muñoz
Entrega N.°	1
Periodo cubierto	24/08/2026 – 01/10/2026

Fecha	Integrante	Herramienta	Tarea apoyada	Prompt utilizado (resumen)	Intervención humana	Resultado y aprendizaje
25/09/2026	Mateo Falla Muñoz	Claude	Selección y análisis de sistemas de inventario existentes	Buscar tres sistemas de gestión de inventario del mercado, preferiblemente de código abierto; describir sus funcionalidades, fortalezas y debilidades; elaborar un cuadro comparativo de funcionalidades, arquitectura y experiencia de usuario, y redactar el análisis y las conclusiones del estado del arte.	Medio	Se conservaron las características de funcionamiento e implementación de los tres sistemas y se filtró la información demasiado técnica o poco relevante. Se aprendió a especificar mejor los requerimientos en los *prompts* y la importancia de verificar los resultados de la IA.
25/09/2026	Luis Alejandro Saraza Marín	ChatGPT	Comparación entre arquitectura monolítica y de microservicios	A partir del enunciado del proyecto, desarrollar la justificación técnica de los microservicios frente al monolito para el sistema, comparando escalabilidad, mantenibilidad, acoplamiento, despliegue independiente, trabajo en equipo y persistencia, de forma breve, con un cuadro comparativo y sin agregar información no respaldada por el documento.	Medio	Se tomó el cuadro comparativo con ajustes y se resumieron manualmente los párrafos. Se aprendió sobre los beneficios de los microservicios para un sistema con muchas funcionalidades y su relación con Docker y Kubernetes.
25/09/2026	Brayan David Collazos Escobedo	Claude	Requerimientos funcionales y no funcionales adicionales	Proponer requerimientos funcionales y no funcionales adicionales a los del enunciado que sean alcanzables en un semestre, sin ser demasiado ambiciosos.	Medio	Se analizó la lista propuesta, se seleccionaron algunos requerimientos y se ajustó su alcance para hacerlos viables en el proyecto (RF15 a RF17 y RNF17 a RNF20).

Fecha	Integrante	Herramienta	Tarea apoyada	Prompt utilizado (resumen)	Intervención humana	Resultado y aprendizaje
27/09/2026	Brayan David Collazos Escobedo	Gemini	Definición del problema y del alcance	A partir de los requerimientos del proyecto, crear una empresa ficticia ubicada en Manizales, con un problema por resolver y el alcance del sistema.	Bajo	El caso propuesto se ajustaba al proyecto; se modificaron los objetivos del alcance que no tenían relación con los requerimientos o eran demasiado ambiciosos para un proyecto académico.
27/09/2026	Brayan David Collazos Escobedo	Claude	Revisión de cumplimiento de la entrega, verificación de fuentes, definición de la pila tecnológica y edición del documento en LaTeX	Analizar el documento de la Entrega 1 frente al enunciado, verificar que todos los entregables estén cubiertos, proponer mejoras, especificar la pila tecnológica usando solo Laravel, Flask y Express, y generar el documento en LaTeX.	Medio	La revisión identificó referencias sin lista bibliográfica, requerimientos sin servicio responsable, requerimientos no funcionales sin métricas y una mención a facturación ajena al alcance. Al contrastar las fuentes se actualizaron las versiones de InvenTree y Part-DB y se retiraron afirmaciones no respaldadas. El equipo definió las tecnologías vistas en clase (MySQL y PostgreSQL, sin *broker* de mensajería), lo que llevó a adoptar el patrón *Outbox* para los eventos.

Reflexión final de la fase

En esta fase delegamos a la IA tareas de exploración y redacción inicial: la búsqueda de sistemas de referencia, los borradores de la justificación arquitectónica y del caso de estudio, la propuesta de requerimientos adicionales y, al final, una revisión de cumplimiento del documento. Lo hicimos porque una primera versión rápida nos permitía discutir sobre algo concreto y dedicar el tiempo a decidir. Aprendimos que la calidad de la respuesta depende de lo específico del *prompt*: cuando no fijamos límites, las herramientas propusieron alcances excesivos y contenido ajeno al caso, como una referencia a facturación que no hace parte del sistema. También detectamos limitaciones concretas: versiones de software que ya habían cambiado al momento de la entrega, citas sin referencia completa y argumentos genéricos que no se ajustaban a El Montañero. Para verificar, contrastamos cada dato de los sistemas analizados con su documentación oficial, revisamos cada requerimiento sugerido frente al enunciado y a nuestra capacidad real en un semestre, y ajustamos las propuestas tecnológicas a las herramientas efectivamente vistas en clase antes de incorporarlas al documento.

Referencias

Apideck. (s. f.). *How to integrate with the Zoho Books API*. Recuperado el 1 de octubre de 2026, de https://apideck.com/blog/how-to-integrate-with-the-zoho-books-api

InvenTree. (s. f.-a). *Docker installation guide*. InvenTree Documentation. https://docs.inventree.org/en/latest/start/docker/

InvenTree. (s. f.-b). *InvenTree: Open source inventory management system* [Repositorio de código]. GitHub. https://github.com/inventree/InvenTree

InvenTree. (s. f.-c). *InvenTree API*. InvenTree Documentation. https://docs.inventree.org/en/0.17.1/api/api/

InvenTree. (s. f.-d). *InvenTree processes*. InvenTree Documentation. https://docs.inventree.org/en/latest/start/processes/

InvenTree. (s. f.-e). *News*. https://inventree.org/news

InvenTree. (s. f.-f). *Part notifications*. InvenTree Documentation. https://docs.inventree.org/en/latest/part/notification/

InvenTree. (s. f.-g). *Parts*. InvenTree Documentation. https://docs.inventree.org/en/stable/part/

InvenTree. (s. f.-h). *Purchasing*. InvenTree Documentation. https://docs.inventree.org/en/latest/purchasing/

InvenTree. (s. f.-i). *Release 1.5.6*. GitHub. https://github.com/inventree/InvenTree/releases/tag/1.5.6

InvenTree. (s. f.-j). *Sales*. InvenTree Documentation. https://docs.inventree.org/en/1.4.x/sales/

InvenTree. (s. f.-k). *Stock expiry*. InvenTree Documentation. https://docs.inventree.org/en/1.3.x/stock/expiry/

Part-DB. (s. f.-a). *Authentication*. Part-DB Documentation. https://docs.part-db.de/api/authentication.html

Part-DB. (s. f.-b). *Concepts*. Part-DB Documentation. https://docs.part-db.de/concepts.html

Part-DB. (s. f.-c). *Kubernetes / Helm charts*. Part-DB Documentation. https://docs.part-db.de/installation/kubernetes.html

Part-DB. (s. f.-d). *OAuth2*. Part-DB Documentation. https://docs.part-db.de/api/oauth.html

Part-DB. (s. f.-e). *Part-DB 2.19.0*. GitHub. https://github.com/Part-DB/Part-DB-server/releases/tag/v2.19.0

Part-DB. (s. f.-f). *Part-DB-server* [Repositorio de código]. GitHub. https://github.com/Part-DB/Part-DB-server/blob/master/README.md

Zoho Corporation. (s. f.-a). *How do I get notified when the quantity of an item drops below the reorder level?* Zoho Inventory Knowledge Base. https://www.zoho.com/us/inventory/kb/items/item-reorder-notification.html

Zoho Corporation. (s. f.-b). *Introduction*. Zoho Inventory API Documentation. https://www.zoho.com/inventory/api/v1/introduction/

Zoho Corporation. (s. f.-c). *Inventory plans and pricing*. https://www.zoho.com/us/inventory/pricing/

Zoho Corporation. (s. f.-d). *Reportes de inventario*. Ayuda de Zoho Inventory. https://zoho.com/es-mx/inventory/help/reports/inventory-reports.html

Zoho Corporation. (s. f.-e). *Replenishment*. Zoho Inventory Help. https://www.zoho.com/us/inventory/help/items/replenishments.html

Zoho Corporation. (s. f.-f). *Reports*. Zoho Inventory Help. https://www.zoho.com/us/inventory/help/reports/reports-overview.html

Zoho Corporation. (s. f.-g). *Users & roles*. Zoho Inventory Help. https://www.zoho.com/au/inventory/help/settings/users.html

Zoho Corporation. (s. f.-h). *Zoho Inventory comes to your pocket with an all-new Android app*. Zoho Inventory Blog. https://www.zoho.com/inventory/blog/zoho-inventory-comes-to-your-pocket-with-an-a

`ll-new-android-app.html`

Zoho Corporation. (s. f.-i). *Zoho Inventory management app* [Aplicación móvil]. App Store. `https://apps.apple.com/app/id1037960494`